

Supplementary Material for PrepBench: A Factorized Benchmark for Cost-Aware Data-Preparation Pipeline Search [Experiment, Analysis & Benchmark]

Jing Chang, Chang Liu, Jinbin Huang, Yu-Xuan Qiu, Rui Mao, Jianbin Qin

This document accompanies the main paper and provides the artifact description, the released file schemas, the exact reproduction procedure, and the hardware/software environment. Every claim in the paper is reproducible from the released files described here. The artifact is available at:

<https://github.com/prepbench2027/prepbench>

1 ARTIFACT OVERVIEW AND CLAIMS SUPPORTED

The artifact is organized around complete run summaries and candidate-level histories, so that every number, table, and figure in the paper can be regenerated offline from released CSVs without re-running the expensive benchmark. The benchmark evaluates nine configurations (seven factorial combinations of the structure/-cost/evaluation/budget axes plus two random controls) across 56 OpenML datasets with five seeds each, at wall-clock budgets of 60 s, 300 s, and 600 s, totalling 2,520 runs per budget. Concretely, the artifact supports the following claims:

- **Mean scores, ranks, and omnibus tests** for the controlled 56-dataset matrix, including the critical-difference diagram, are regenerated from `experiment_records.csv`.
- **Pairwise Wilcoxon and TOST equivalence tests** are regenerated from `experiment_records.csv` and the derived paired per-dataset score tables.
- **Zero-penalty sensitivity** is regenerated from `results/sensitivity/` `↪` and supports the claim that the random-control ranking is not explained by the default estimated-cost or graph-complexity penalties.
- **Stronger guided-EA sensitivity** is regenerated from `results/` `↪` `sensitivity/strong_ea300/` and supports the scoped claim that optimizer strength matters while the combined cost-aware variant remains below the plain DAG in final accuracy.
- **Random-space fairness audit** is regenerated from `random_fairness_audit` `↪` `.csv` and records candidate-level structure, workflow uniqueness, and runtime diagnostics for the zero-penalty runs.
- **Paired budget deltas** and the budget-scaling figure are regenerated from `experiment_records.csv`.
- **Evaluation-allocation counters** are regenerated from `experiment_records` `↪` `.csv`.
- **Staged-screening diagnostics** and the calibration scatter plot are regenerated from `search_history.csv`.
- **Cost-model calibration**, including the static-cost vs. runtime scatter plot moved out of the main paper for space, is regenerated from `search_history.csv`.
- **Incumbent convergence** is regenerated from `search_history.csv`.
- **Efficiency frontier** is regenerated from `experiment_records.csv`.

- **Provenance and robustness**, including the completed-evaluation-only block summarized in the main paper, come from coverage- and trace-audit analyses over `experiment_records.csv` and `search_history` `↪` `.csv`.
- **External CtxPipe reference** is regenerated from `ctxpipe_scores` `↪` `csv` and `ctxpipe_paired_comparison.csv`; it is contextual evidence, not a controlled ablation. Raw results of CtxPipe are in `ctxpipe_results` `↪` `tsv` and `ctxpipelines.json`.

2 REPOSITORY LAYOUT

```
prepbench-vldb-eab-artifact/  
  README.md  
  requirements.txt  
  supplementary_prepbench.pdf  
  paper/  
    paper-vldb.pdf  
  code/  
    baselines/  
    parallel_evolutionary_pipeline/  
    data_loader/  
    experiments/  
      run_matrix.py  
      analyze_icde_eab_extras.py  
      summarize_sensitivity_runs.py  
      make_paper_result_bundle.py  
    figure_scripts/  
    scripts/  
  results/  
    main_matrix/  
      eab60/ eab300/ eab600/  
        experiment_records.csv  
        run_metrics.csv  
        search_history.csv  
        run_manifest.json  
        paper_result_bundle/  
      sensitivity/  
        zpen300/ zpen600/  
        strong_ea300/  
        zero_penalty_audit/  
          random_fairness_audit.csv  
        strongEA_compare/  
        sensitivity_summary_by_config.csv  
        sensitivity_paired_tests.csv  
    external_baselines/  
      ctxpipe_scores.csv  
      ctxpipe_paired_comparison.csv  
      ctxpipe_results.tsv  
      ctxpipelines.json  
  figures/  
    data/  
      manifest.csv
```

3 RELEASED FILE SCHEMAS

Every released CSV is documented below; column names match the analysis scripts exactly.

experiment_records.csv (*per budget*). One row per (dataset, seed, configuration, budget) run — the unit used for all score-based statistics:

```
dataset, config, seed, budget, score,
score_source in {full_candidate, stage1, fallback},
estimated_cost, serial_cost, critical_path_cost,
runtime_seconds, branches, nodes, depth,
n_full_evals, n_screened_out, n_pruned
```

run_metrics.csv (*per run directory*). Augmented per-run metrics used for anytime, efficiency, and allocation analyses:

```
dataset, config, seed, budget, score,
n_full_evals, n_ok_evals, n_screened_out, n_pruned,
branches, nodes, depth, time_to_target, anytime_auc
```

search_history.csv (*per run directory*). One row per candidate event (the trace used for allocation, screening, and cost-calibration diagnostics):

```
dataset, config, seed, budget, candidate_index,
status in {ok, screened_out, pruned, timeout, failed, fallback},
score_source, score, stage1_score, full_score,
best_so_far, improved_incumbent,
estimated_serial_cost, estimated_critical_path_cost,
elapsed_seconds, branches, nodes, operator, generation
```

summary_by_config.csv. Per-config aggregate statistics across datasets and seeds:

```
budget, config, mean_score, median_score, std_score,
mean_evals, mean_ok_evals, mean_screened_out, mean_pruned,
mean_branches, mean_nodes, mean_depth,
mean_time_to_target, mean_anytime_auc, n_runs
```

run_manifest.json. Experiment provenance: budget, list of seeds, list of configs, list of datasets, penalty settings, population/offspring settings when changed, and the run tag used to separate main-matrix and revision-sensitivity runs.

sensitivity/ files. Sensitivity outputs support the optimizer/objective checks added to the main paper:

- sensitivity_summary_by_config.csv: aggregate means, medians, ranks, and run counts for zero-penalty and stronger-EA runs.
- sensitivity_paired_tests.csv: paired per-dataset Wilcoxon comparisons after averaging seeds.
- random_fairness_audit.csv: candidate-level audit of workflow uniqueness, mean nodes, mean branches, runtime, success rate, and entropy diagnostics for the random and guided DAG variants.
- Per-run experiment_records.csv, run_metrics.csv, and search_history. $\hookrightarrow$ csv under each sensitivity run directory.

trace_summary/ files.

- status_summary.csv: distinct status values $\times$ config $\times$ budget.
- per_run_incumbent.csv: one row per (dataset, config, seed, budget) recording incumbent score, score_source, status counts, and evaluation counts.
- staged_metrics.csv: per (config, budget) Pearson and Spearman correlation between stage-1 and full score, promotion rate, and incumbent-threat rate.
- stage_full_sample.csv: reservoir sample of (stage1_score, full_score) pairs for scatter plotting (Fig. 4).

paper_result_bundle/ files. Generated by make_paper_result_bundle.py:

- stat_tests.csv: Friedman omnibus test and Wilcoxon signed-rank pairwise tests (each configuration vs. the DAG baseline D-S-F-A, and the random controls vs. the structured configurations).
- tost_sensitivity.csv: mean/median differences and within-margin flags for $\delta \in \{0.005, 0.01, 0.02\}$ against D-S-F-A.
- ctxpipe_scores.csv and ctxpipe_paired_comparison.csv: CtxPipe per-dataset reference scores and paired comparison against the controlled references.
- coverage_audit.csv: per-budget expected vs. observed runs at the run level and the candidate-trace level.
- trace_audit.csv: cross-validation between run_metrics and search_history $\hookleftrightarrow$ key sets.
- dataset_config_scores.csv, dataset_config_seed_scores.csv: per-dataset aggregated scores.

manifest.csv. The manifest has one row per dataset and records the canonical dataset name, the artifact-relative CSV path, the target column, the expected info.json path, the original DeepLine source URL, and the dataset statistics used in Table 2. The OpenML-id fields are retained for compatibility with external-baseline scripts but are left blank when the artifact uses the DeepLine CSV source directly; in that case csv_path and target are the authoritative loading fields.

Provenance contract. For every run with at least one completed full evaluation, the reported score in experiment_records.csv equals the score of the best incumbent in search_history.csv. The run-level allocation counters (n_full_evals, n_screened_out, n_pruned) match the aggregated candidate-status counts. Runs whose reported score has score_source = fallback (i.e., no full candidate evaluation completed) are excluded from the completed-evaluation-only robustness block.

4 REPRODUCTION PROCEDURE

The released results reproduce all paper numbers in minutes on a laptop; the expensive end-to-end benchmark is optional and supports resumption.

```
# From the artifact root:
cd prepbench-vldb-eab-artifact
conda create -n prepbench python=3.12
conda activate prepbench
pip install -r requirements.txt
```

```
python code/experiments/make_paper_result_bundle.py \
--results results/main_matrix \
--out /tmp/prepbench_result_check
```

```
python code/figure_scripts/cd_diagram_600.py
python code/figure_scripts/incumbent_convergence_600.py
python code/figure_scripts/budget_delta_score_vs_evals.py
python code/figure_scripts/staged_calibration_dctb_600.py
python code/figure_scripts/efficiency_frontier_600.py
python code/figure_scripts/cost_runtime_calibration.py
```

```
# Regenerate revision-sensitivity summaries
python code/experiments/summarize_sensitivity_runs.py \
--root results/sensitivity \
--out /tmp/prepbench_sensitivity_check
```

These scripts read only the released CSVs under results/ and write PDFs to figures/. They are deterministic and produce the figure panels used in the paper and the supplementary diagnostic plots.

```
# Re-run the full benchmark (optional)
python code/experiments/run_matrix.py \
  --budget 600 --seeds 42 114 256 512 768 \
  --out results/main_matrix/eab600 \
  --processes 32 --per-eval-timeout 10 \
  --run-tag main_600

# Re-run a zero-penalty sensitivity block
python code/experiments/run_matrix.py \
  --budget 600 --seeds 42 114 256 512 768 \
  --configs D-Rand D-S-F-A D-C-T-B \
  --out results/sensitivity/zpen600 \
  --processes 64 --per-eval-timeout 10 \
  --fitness-cost-penalty 0 \
  --fitness-complexity-penalty 0 \
  --run-tag zero_penalty_600

# Re-run the stronger guided-EA sensitivity block
python code/experiments/run_matrix.py \
  --budget 300 --seeds 42 114 256 512 768 \
  --configs D-S-F-A D-C-T-B \
  --out results/sensitivity/strong_ea300 \
  --processes 64 --per-eval-timeout 10 \
  --fitness-cost-penalty 0 \
  --fitness-complexity-penalty 0 \
  --population-size 40 \
  --offspring-size 40 \
  --run-tag strong_ea300
```

This reproduces the raw runs and writes fresh `experiment_records.csv`, `run_metrics.csv`, and `search_history.csv`.

5 STATISTICAL PROTOCOL DETAILS

For completeness we record the exact procedures used in the paper.

- **Aggregation.** Scores are averaged over the five seeds within each (dataset, configuration) cell, giving a complete 56×9 block per budget; the dataset is the unit of analysis.
- **Omnibus + post-hoc.** A Friedman test per budget; on rejection, the Nemenyi post-hoc test with $k = 9$, $N = 56$, $q_{0.05} = 3.102$, giving $CD = 1.605$.
- **Pairwise.** Wilcoxon signed-rank on paired per-dataset scores, with Holm–Bonferroni family-wise control within each budget.
- **Equivalence.** Two one-sided tests (TOST) on paired per-dataset differences. The main margin is $\delta = 0.01$; the sensitivity analysis sweeps $\delta \in \{0.005, 0.01, 0.02\}$ in the equivalence-margin sensitivity table. Two configurations are declared equivalent only if both one-sided hypotheses reject at $\alpha = 0.05$.
- **Seeds.** $\{42, 114, 256, 512, 768\}$.

6 HARDWARE AND SOFTWARE ENVIRONMENT

All runs were produced on a single fixed machine; the fingerprint is summarized in Table 1. Wall-clock budgets (60 s / 300 s / 600 s) are interpreted on this machine; the 60 s $\rightarrow$ 300 s plateau (RQ3) is a property of this cost regime.

7 LICENSE AND DATA AVAILABILITY

All datasets used are public DeepLine tabular classification datasets; per-dataset details are listed in Table 2. The file `data/manifest.csv` maps each canonical dataset name to its artifact-relative CSV path, target column, and original DeepLine source URL. If the raw CSV cache is not bundled, users can download the public DeepLine CSVs listed in

Table 1: Environment fingerprint.

Field	Value
CPU	AMD EPYC 7543 32-Core Processor (dual socket)
Logical threads	128 (SMT)
Memory	512 GB
OS	Ubuntu 20.04.6 LTS
Parallel backend	joblib (loky)
Python	3.12.9
scikit-learn	1.6.0
numpy	1.26.4
scipy	1.16.3
pandas	2.3.3
dataclasses	0.6
matplotlib	3.10.1
python-dotenv	1.2.1

the manifest and arrange them as `data/deepline_dataset/<dataset>/data.csv` with the corresponding `info.json` target metadata. A local cache can be selected with the `PREPBENCH_DATA_DIR` environment variable.

Table 2: Details of the 56 datasets used for the evaluation: number of instances, number of features, percentage of categorical features, percentage of missing values, and number of labels in the target (number of classes). Dataset names match `data/manifest.csv`, the released result CSVs, and the main-paper figures.

Dataset	# Inst.	# Feat.	% Categ.	% Miss	# Labels
Accident_Casualties	7500	14	0	0	4
adult	7480	15	60	0	2
analcata_data_broadwaymult	285	8	50	1.2	7
analcata_data_germangss	400	6	66.7	0	17
ar4	107	30	0	0	2
bank-full	7368	15	53.3	0	2
baseball	1340	17	5.9	0.1	3
biodeg	1055	42	2.4	0	2
blood-transfusion	748	5	0	0	2
bodyfat	252	15	6.7	0	2
braziltourism	412	9	0	2.6	7
bureau	7125	302	5.3	30.6	2
car	1728	7	100	0	4
chatfield_4	235	13	7.7	0	2
cmc	1473	10	0	0	3
crx	690	16	62.5	0.6	2
data_banknote_authentication	1372	5	0	0	2
dermatology	365	35	0	0.1	6
digggle_table_a2	310	9	11.1	0	2
disclosure_z	662	4	25	0	2
Frogs_MFCCs_family	7195	23	4.3	0	4
Frogs_MFCCs_gen	7195	23	4.3	0	8
Frogs_MFCCs_spec	7195	23	4.3	0	10
glass	214	10	0	0	6
haberman	306	4	0	0	2
home_credit	7250	343	0	19.3	2
HTRU_2	7000	9	0	0	2
imagesegmentation	2310	20	5	0	7
IndianLiverPatientDatasetILPD	583	11	0	0.1	2
Iris	150	5	20	0	3
kc3	458	40	0	0	2
kidney	76	7	42.9	0	2
LendingClubIssuedLoans	7370	53	22.6	3	12
magic04	7450	11	9.1	0	2
mammographic_masses	961	6	0	2.8	2
movement_libras	360	91	0	0	15
no2	500	8	12.5	0	2
plasma_retinol	315	14	28.6	0	2
pm10	500	8	12.5	0	2
schizo	340	14	21.4	17.5	2
Skin_NonSkin	7122	4	0	0	2
socmob	1156	6	83.3	0	2
solar-flare	1066	13	23.1	0	6
test_bng_cmc	55296	10	0	0	3
test_breast	699	10	10.2	0	2
test_credit	1000	21	66.7	0	2
test_eucalyptus	736	20	30	3	5
test_ilpd	583	11	9.1	0	2
test_irish	500	6	66.7	1.1	2
test_phoneme	5404	6	0	0	2
The_broken_machine	7490	59	9.4	2	2
triazines	186	61	1.6	0	2
veteran	137	8	12.5	0	2
weatherAUS	7357	25	24	9.1	2
wilt	4839	6	16.7	0	2
Wine_classification	178	14	0	0	3